

Sonder

A trip-planning system built by someone tired of trip-planning systems

1. Abstract

I've been planning my own trips for fifteen years and I can tell you exactly what every existing platform is bad at. TripAdvisor optimises for the lowest common denominator, so the top three things to do in any city are the same things every coach tour stops at. Google Travel knows my budget but not my mood. Instagram knows my mood but not whether the place I just saved is closed on Tuesdays. Airbnb knows where I'll sleep but stops there. And none of them — not one — solves the problem of *who I'll be there with*. Solo travellers get nothing. Couples get hotel suggestions. Anyone who's ever wanted to share a trip with a stranger they actually click with is on their own, defaulting to hostel bars and dating apps repurposed badly.

Sonder is what I wanted to exist. It does three things no existing product does at once: it generates a day-by-day itinerary that names *real* places — the café with the rosé-coloured awnings on Largo do Carmo, not "explore the old town" — grounded in a hand-curated corpus of destinations and activities I'd actually recommend; it matches solo and couple travellers against a measurable compatibility surface rather than swiping over bios; and it sustains a believable social layer through cold-start by seeding a population of LLM-designed synthetic travellers who post, open trips, and message real users about their plans.

The engine underneath is a multi-provider language model stack — Anthropic primary, OpenAI fallback, with separate tiers for fast persona voice (Claude Haiku) and deep generation (Claude Sonnet), a dedicated validator tier, plus image generation, voice synthesis, and a 1536-dimensional embedding space holding everything from city contexts to traveller personalities. The whole thing runs through a streamed pipeline where the user sees day one of their itinerary within fifteen seconds while later days are still being written, and a validator engine that catches assistant-voice leakage, hallucinated venues, and broken persona consistency before they hit the screen.

What it delivers: itineraries with zero negative-constraint violations against ten-to-twenty-five percent on a prompt-only baseline; persona-grounded chat replies in under two seconds at one-twenty-fourth the cost of a large-model approach; a co-traveller match score calibrated so a 0.6 means roughly a sixty percent chance the other side accepts — not the uniformly high cosine numbers that make compatibility scoring meaningless on every other matching product I've used.

This report covers the data the system runs on, the models choosing it, the architecture deploying it, and the integration battles I fought to get there.

2. The data underneath everything

2.1 Four planes of data

Sonder isn't a model I trained on a static dataset. It's a live system that produces and consumes data across four planes simultaneously, and the architecture decisions that mattered most came from understanding where each plane lived and what it was good for:

- **The reference corpus** — destinations and activities I curated by hand, embedded once, queried thousands of times. Roughly fifty cities, around five hundred activities. Hand-curated because every existing scraped travel corpus reads like a search engine result page.
- **The synthetic traveller population** — 192 solo travellers and 18 couples I had a language model design under tight diversity constraints. Each one has a written backstory, a voice anchor, two or three quirks, an emotional signature, and a stylised portrait. They live in the same vector index as the destinations.
- **The operational data plane** — what real users produce as they use the product. Profiles, generated itineraries, chat sessions, journal entries, social posts, shared-itinerary negotiation history.
- **Telemetry** — every language model call instrumented, every validator outcome logged, every match score recorded with its feature breakdown. This is the substrate for phase-two ranker learning, and it's the only reason I sleep at night about LLM regressions.

2.2 The frameworks I'm standing on

I didn't invent the psychology of travel. Three pieces of prior work shaped Sonder's foundations and deserve naming explicitly:

Push-Pull Motivation Theory. Originally introduced into travel research by *Dann (1977)* and extended by *Crompton (1979)*, the theory frames travel as the product of two distinct motivational forces — what pushes you away from home (the intrinsic side: escape, novelty, connection, reflection, curiosity, milestone) and what pulls you toward a specific destination (the extrinsic side: nature, culture, food, nightlife, comfort, local immersion). I implemented this as a twelve-dimension persona space — six push dimensions, six pull dimensions — each defined by a substring-matched keyword set rather than a single trigger word. So “*out of comfort zone*” counts as one signal toward *adventure_novelty*, not three accidental hits across unrelated dimensions. The whole point of the keyword-list-not-single-word approach is auditability: when the system tells a user they score high on *escape_reset*, I can point at the exact phrases in their input that fed that score. No black-box psychology assignments.

Push (why they travel)	Pull (what they want from the place)
<i>escape_reset</i> — disconnect, recharge, leave routine	<i>nature_outdoors</i> — landscapes, weather, physical settings
<i>adventure_novelty</i> — push themselves, first-time experiences	<i>culture_history</i> — heritage, museums, local arts
<i>connection</i> — share with the people they love	<i>food_drink</i> — local cuisine, regional specificity
<i>reflection</i> — process, gain perspective	<i>nightlife_social</i> — bars, clubs, live music

Push (why they travel)	Pull (what they want from the place)
<i>curiosity</i> — go deeper than the guidebook	<i>comfort_luxury</i> — refined service, high-end stays
<i>prestige_reward</i> — milestone trips, dream destinations	<i>exploration_local</i> — neighbourhoods, daily-life immersion

GoEmotions. *Demszky et al. (2020), arXiv:2005.00547* — a 58,000-row dataset of Reddit comments labelled across 27 fine-grained emotion categories. I don't ship the 58,000 rows. What I use is the label vocabulary itself. Each of the 27 emotions gets a one-line tone-anchored gloss — *realization* is "a quiet click — coming to understand something", not a dictionary definition — and I embed those 27 glosses once at process start using the same 1536-dimensional embedding model that powers the rest of the system. A user's free-text input then gets classified by cosine similarity against those 27 anchor vectors. Defensible scores, no separate classifier to retrain, no model registry to maintain, lives in the same vector space as everything else. The tone-anchored gloss choice is deliberate: dictionary-style definitions cluster too tightly in the embedding space; tone anchors crisp it.

An eight-key emotional signature taxonomy I synthesised specifically for travel. This is the third layer — coarser than GoEmotions, more identity-shaped than PPM — covering eight traveller archetypes: *story_collector*, *reset_seeker*, *aesthetic_pilgrim*, *depth_diver*, *energy_chaser*, *ritual_keeper*, *quiet_observer*, *threshold_walker*. The inferrer picks one key per user based on their GoEmotions distribution plus their structured persona answers, attaches a confidence level, and the signature then becomes private framing for every persona-voiced surface. The key itself is never shown — only the derived emotional tone phrase like "soft afternoon energy" surfaces in the UI. This was the hardest naming exercise in the entire project. I went through four iterations of the taxonomy before landing on these eight. The earlier versions had too much overlap; the final set was tested against synthetic personas to make sure each archetype produced a meaningfully distinct chat-reply register before I locked it.

So the personality stack is three lenses at different granularities: 27-label fine emotion → 12-dimension push-pull motivation → 8-key voice signature. Each consumer in the system chooses the lens appropriate to its task. The chat reply prompt reads the 8-key signature. The matcher reads the 12-dimension PPM space. The persona reveal copy uses all three.

2.3 The travel APIs I integrated and why each one fought back

A travel product without real data about real places is fiction. I plumbed five external APIs into Sonder, and every single integration had a failure mode I had to engineer around:

Wikipedia REST API — destination overview lede paragraphs and the article's primary image. The integration looked trivial until I tried it on a region rather than a city. Wikipedia's REST endpoint returns whatever the article's infobox image is, and for "Patagonia" that's a map of southern South America with the region shaded gold. I had a beautiful cinematic reveal screen ready to ship, and what was rendering as the hero photo on every region-shaped query was a Wikipedia location map. The fix took an evening: a URL-substring rejection filter that drops any `.svg` (almost always maps, flags, or coats of arms) plus any URL containing *map*, *karte*, *location*, *locator*, *satellite*, *topographic*, *flag_of*, *coat_of_arms*, or *seal_of*. Combined with a 14-day client-side cache invalidated by a key bump so users with cached maps refetched fresh on next load.

Pixabay — the fallback when Wikipedia returns nothing usable, and the source for the cinematic destination reveal montage. Required server-side proxying because exposing the API key client-side would burn rate limits via abuse. Required a country-less retry path because *"Patagonia Argentina"* sometimes returns zero hits while *"Patagonia"* returns thirty. Required an in-process cache keyed by (query, count) so refreshing the reveal screen doesn't re-pay quota for the same five photos. The Pixabay multi-photo lookup is what powers the Ken-Burns cinematic montage — five photos cycling every 1.2 seconds during the destination reveal, crossfading with a slow zoom — and the cache makes that feel free on every subsequent view.

OpenWeather — current conditions by lat/lon. Cheap, fast, well-behaved.

Nominatim (OpenStreetMap) — geocoding city/country tuples to coordinates. OSM's terms of service request no more than one request per second from a single source, and I wanted to respect that rather than risk getting rate-limited mid-traffic spike. So Sonder wraps Nominatim in a process-wide token-bucket rate limiter and a 30-day disk cache. A repeat user planning a known destination hits the network for essentially nothing on subsequent loads.

ExchangeRate API — currency conversion at the input boundary. All internal cost fields in Sonder are USD; conversion happens once when the user submits a budget in their preferred currency. The integration came with a 3-second timeout and a hardcoded fallback table for thirty currencies so the product doesn't break when ExchangeRate's free tier flakes mid-night.

A pattern emerged across all five integrations: **cache aggressively, fail soft, never let a partner outage break the product**. Wikipedia is down? Use Pixabay. Pixabay is down? Fall back to a gradient hero. OpenWeather slow? Skip the weather line. Nominatim rate-limited? Cache hit. ExchangeRate offline? Hardcoded rates from a year ago. The user never sees an error screen because a third-party API decided to have a bad afternoon. This is non-negotiable for a product where the user is mid-flow planning the trip of their year.

2.4 The synthetic traveller corpus — why I built a population from scratch

The matching surface needed a candidate pool. Real users wouldn't exist on day one. The honest options were: (a) launch with an empty matching screen and pray for organic adoption to fill it, or (b) seed a population. I chose (b), but the way I did it matters.

The diversity matrix is locked: sixteen cities across five continents (New York, Mexico City, Buenos Aires, Bogotá, London, Paris, Berlin, Lisbon, Istanbul, Lagos, Cape Town, Dubai, Mumbai, Bangkok, Tokyo, Seoul), three age buckets (20-30, 30-40, 40-50), two genders (50/50 hard-locked), two personas per cell. That's 192 solo travellers. A separate matrix produces 18 male-female couples for couple-mode matching. The cell-density choice came after a real product failure I'll detail in section 3.

Each persona is generated through a **two-stage blind-writer pipeline**. The first language model gets only what a novelist would get — city, age bucket, gender, and four persona question option keys to pick from. It writes the character: name, voice anchor (their first-person reference sentence), small-thing answer, quirks, appearance descriptor. **It never sees the PPM dimensions, the emotional signature taxonomy, or any**

of the matching feature names. The second stage — the inferrer — then runs the written persona through exactly the same pipeline a real user would go through, assigning PPM dimension labels and an emotional signature based on what was written, blind to what the writer was "supposed" to produce.

This is the most important design decision in the entire system, and it's why Sonder's personas don't read as obviously machine-generated. Information starvation as quality control. An LLM with the answer key in front of it writes to that key. An LLM that has to guess writes something honest. The portraits are generated by gpt-image-1 from the appearance descriptor only — painterly, explicitly not photoreal, biased that way to support the "*Sonder Curated*" disclosure pattern. The cost is roughly \$2-4 in API calls per seed run. Worth every cent.

2.5 Schema realities — what's actually in the data

Free-text length distributions across the synthetic corpus:

Field	Median (words)	Range
Voice anchor	20	8-40
"Small thing" free text	14	6-35
Quirks (concatenated)	16	8-30
Embedding text (the full string vectorised)	110	40-220

Two missing-value paths bit me in production and are worth surfacing:

Per-question answer salience. A per-user weighting that boosts the matching contribution of free-text answers a user revealed more about themselves on. When absent — older profiles created before I added the field — the matcher falls back to uniform weighting. Visibly different match scores between users on the same candidate pool. I had to write a fail-open path in the ranker so older profiles still get matches, and the scores are honestly worse, but the product doesn't dead-end them.

Gender on synthetic personas. This one was a near-disaster. After I shipped the same-gender hard filter for solo travellers as a safety default, the matching surface immediately started returning zero candidates for everyone. The reason: I had added gender to the synthetic persona schema, but the Pinecone metadata write step in the seed script — which I'd written months earlier — was only storing gender in the log-preview output, not the actual metadata dict. So every record in production had no gender field. The filter saw zero qualifying candidates and the safety default was "fail open to mixed matching" — which meant the gender filter was never actually firing. I diagnosed this at 2am, wrote a metadata-only patch script that rebuilt the deterministic diversity matrix and called `index.update(set_metadata={"gender": ...})` per profile id without re-paying the language model or image generation cost, ran it against production, watched 96 records get patched in under a minute. Then bumped the seed-script code path so future re-seeds wouldn't repeat the bug.

2.6 The drift patterns and how the system fights them

Two recurring failure modes surfaced in generated content during the first weeks of running the synthetic-agents loop:

Sales-register drift. The early synthetic open-trip notes read like travel marketing copy. *"Join me for an unforgettable adventure!" "Looking for like-minded wanderlust souls." "Open to fellow travel buddies."* You know the voice. It's the voice that makes you trust nothing on a travel platform. The fix was to put *forbidden-openers* lists with explicit bad examples in every persona-voiced system prompt. Show the model exactly what bad looks like; don't just describe it.

Question-loop drift in chat. Personas would start interrogating users after about turn seven of a chat. *"What are you most excited about? What's your favourite memory of travelling? What would your dream day in Lisbon look like?"* Real people don't text like this. The fix was a runtime instruction inserted into the prompt when two or more of the persona's recent turns had ended with a question mark — a *"breathe hint"* that tells the model to react, observe, or share something small instead. Two-line code change, completely changed the chat register.

Typography drift caught the worst. A real user pointed out a message that read *"the street pad thai is genuinely different from restaurant versions , way more char on the noodles. there's a few stalls near the floating markets..."* Two errors in one sentence: stray space before the comma, singular contraction with a plural subject. The first one fixes universally — a regex collapses `\s+` before punctuation in every persona-voiced message before broadcast. The second one I added to the system prompt directly under a new *"casual ≠ sloppy"* rule: *"there are a few stalls", not "there's a few stalls". Texting register means short and casual, not broken. A real person texting still hits agreement and spacing.*

2.7 The five hallucination categories the validator watches

Every persona-voiced surface in Sonder runs through a validator stack that watches for five regression types:

Category	What it catches
Assistant-voice leakage	<i>"How can I help you?", "I'd be happy to..."</i> — chatbot register bleeding through
AI / tooling leakage	<i>"As an AI", "I'm a language model"</i>
Memory contradiction	Persona claims to have been somewhere, then later denies it within the same chat
Token-level failure	Empty generations, repetition stutters, malformed JSON
Internal-taxonomy leakage	Persona uses the system's labels (<i>push, pull, motivation</i>) instead of behaviour

Each category has a deterministic local pre-check that runs first — regex-based, instant, free — and a critic-prompt fallback that runs after if the local check is inconclusive. The combination is important. It kills the cheap failures pre-LLM and reserves model spend for the genuinely ambiguous cases. Telemetry tracks first-try approval rate, repair-triggered rate, and a continuous semantic-genericity score against a fourteen-stem set (*"sounds amazing", "hidden gem", "bucket list", "fellow traveler"*). The genericity score is particularly useful because it's continuous. A creeping rise in mean genericity over weeks is a leading drift

indicator before any individual reply looks obviously bad.

3. The systems I built and what I compared them against

I'm picking three problems to walk through in detail, each architecturally different from the other two: a retrieval-grounded generation problem, a validator-gated conversational generation problem, and a feature-pipeline ranking problem. For each I had a champion approach in production and a documented baseline I evaluated against.

3.1 Itinerary generation — full RAG pipeline against pure prompt LLM

What the user is trying to do. Punch in a destination, dates, a budget, free-text answers about why they're travelling, plus must-haves and things to avoid. Get back a day-by-day itinerary that names *specific* places, respects every negative constraint, fits their pace and budget, and tells them *why this particular activity for you*.

My approach. Retrieval-augmented generation grounded in the curated venue corpus. The user's persona gets inferred across three pipelines running in parallel — a text embedder, the GoEmotions cosine classifier over their free text, and a language model that picks PPM dimension labels from a closed tool-use enum. None of the three sees the others' outputs while running. The destination gets selected and ranked from the city vector store. Activities for that destination get retrieved from the activity vector store, ranked through a feature pipeline (cosine score, persona-question salience-weighted overlap, ordinal pace fit, budget feasibility, interest overlap), then handed to the itinerary-generating language model.

The model receives: the user's persona, the retrieved real activities, the emotional signature framing. The model never sees: the ranker weights, the cosine scores, any other user's persona, the validator's prompt. It's given exactly the slice of context it needs and nothing else. Outputs stream day-by-day — Day 1 renders on the screen within fifteen seconds while Day 7 is still being generated. The streaming pattern parses JSON forward-only and yields a parsed day the moment its closing brace lands, so the user is reading and the model is still writing.

A separate validator engine then critiques the result across seven categories — budget fit, pacing realism, must-haves covered, avoid-list respected, day-sequence logic, activity specificity, feasibility risk. Outputs that fail can go through a refinement loop, up to three regeneration attempts, each one re-embedding the persona with the failure feedback baked in so the model gets a fresh query rather than grinding on the original.

The challenger. Same persona and constraints rendered into a single prompt. Language model writes the itinerary cold. No retrieval, no ranker, no validator, no refinement, no streaming. The lazy version.

What I measured.

Dimension	Champion (RAG + rank + validator)	Challenger (prompt only)
Specificity — named venues, neighbourhoods, times of day	High	Low — generic "old town", "have a nice dinner"
Must-haves coverage	96-100% (validator-gated)	60-75%
Avoid-list violations	~0% (deterministic pre-filter on retrieval)	10-25% — model invents plausible venues that violate the negative list
Budget adherence	Hard pre-rank filter, never violated	30-40% violation rate on luxury-tier prompts
Validator first-try approval	~78%	Not applicable
Time to first day visible	12-18 seconds (streamed)	45-90 seconds (whole-buffer wait)
Hallucinated venue rate	Rare — model grounded on real activities	Common — plausible-sounding fictional places
Output token cost per trip	~6-8k tokens (large tier) plus retrieval	~6-10k tokens (large tier)

The integration challenge that nearly killed this. The user-initiated revise flow used to call the same three-attempt refinement loop as the initial generation pipeline. Each iteration is a full regen plus validate. The loop was designed for the orchestrator's quality gate, not for revision. Wall time on a revise was hitting two to three minutes, and Cloudflare's edge function proxy was killing the connection somewhere around thirty seconds, causing the page to hang with no error and no result. I rebuilt the revise path as a single-pass classifier-routed pipeline with SSE streaming so days appear as they parse. Then routed small-scope edits to a day-targeted prompt that regenerates only the affected days instead of the whole trip. Average revise dropped from ~150 seconds to ~25 seconds.

Trade-offs. The champion approach requires ongoing curation of the destination and activity corpora — fifty cities and five hundred activities is a curation budget, not an infinity. When a user picks a city not in the corpus, the system falls back to "invent plausible activities for {city}" mode, which re-introduces the hallucination risk of the challenger. Automated corpus expansion is real future work and not a v1 problem. The challenger is operationally trivial but the outputs are generic, which is precisely what every existing recommender already produces. Sonder exists to not be that.

3.2 Chat replies — small-tier with validator-repair against large-tier prompt

What the user is trying to do. Text a synthetic persona inside Sonder. Get a reply in character — texting register, no assistant voice, no AI leakage, no semantic genericity, no contradiction of what the persona said three turns ago — within a perceived-real latency budget of under five seconds.

My approach. Route chat replies to the small language-model tier (Claude Haiku, or GPT-4o-mini under fallback). The persona's prompt layers three private framing blocks before the style rules. First, a hard trip-scope block — *"the trip is to Lisbon; never mention your home city, never suggest alternative destinations"* — which prevents the model from drifting back to wherever its training data has more mass.

Second, a private psychology block where PPM dimensions and the emotional signature are passed as framing with an *explicit instruction never to name the words push, pull, motivation, alignment, or friction in output*. Third, a per-turn breathe hint when recent turns have been too question-heavy.

The three pre-LLM fetches needed to assemble the context — vector store candidate lookup, itinerary fetch, message history — run in parallel rather than serial. That alone saved 600-800ms.

Here's the trick that makes the whole thing work: **the reply broadcasts to the user immediately, before validation**. A separate validator task runs asynchronously after broadcast. It first does a deterministic local pre-check (minimum reply length, repetition detection, semantic genericity score against the fourteen-stem filler set). If issues fire, a repair prompt rewrites the reply in a single pass under a 50-word ceiling. If the repair actually changes the text, the persisted message gets updated and the chat surface receives a *message-edited* event that swaps the bubble text in place.

The user sees the reply land instantly. They watch it quietly self-correct a moment later when needed. Quality without latency cost. This is the single biggest UX win in the system and the pattern generalises to any conversational surface where small-model-first plus async-repair outperforms a single large-model call.

The challenger. Same persona system prompt routed to the large tier with no validator and no edit-in-place.

Dimension	Champion (small-tier + validator)	Challenger (large-tier, no validator)
Median time to user-visible reply	~1.4 seconds	~4.2 seconds
First-try validation pass rate	~74%	N/A
Repair-triggered rate	~26%	N/A
Banned-filler emission after cleanup	<1%	12-18% ("oh nice", "honestly", "love that")
Token-level failures (empty, stutter)	0% (local pre-check kills pre-LLM)	1-2%
Persona consistency across 20 turns	High	Medium — drifts toward generic supportive register
Cost per reply	~\$0.0005	~\$0.012

Three times faster, twenty-four times cheaper per reply, and the user doesn't experience the small tier as small.

3.3 Co-traveller matching — feature pipeline with learning against pure cosine baseline

What the user is trying to do. Get a top-three list of potential co-travellers ranked by a number that honestly means *probability we'll click*, not a uniformly high cosine that hides everyone in a narrow band.

My approach. A stage-based ranking engine that runs the same code path across three matching surfaces — co-traveller, destination, activity — with each surface declaring its own policy (which features to score, what weights, what feedback hyperparameters). Ten reusable scoring functions are available to any policy: raw cosine retrieval score, persona-question overlap weighted by the user's per-question answer salience, emotional-signature exact match, pace ordinal distance, budget ordinal distance, travel-style exact match, two flavours of interest overlap, activity cost fit, pace-duration fit.

Hard pre-ranking filters fire before scoring. Budget feasibility — raw per-day budget cutoff, no fudge multipliers, because if you said your budget is \$80/day you didn't mean \$200/day. Avoid-list veto. Travel-style hard filter so couples never see solo personas. **Same-gender hard filter for solo travellers** so solo women match women and solo men match men. That last one is a safety default for cold-strangers matching and was the source of the biggest production firedrill in the entire project — detailed in a moment.

Per-user learning runs through two paths. The explicit path: free-text revise feedback gets keyword-mapped to specific features (saying "*cheaper*" boosts the budget-fit weight going forward), with decay so repeated similar feedback dampens rather than oscillates. The implicit path: a sarcasm-aware, negation-aware signal scanner runs after every chat message, extracts compatibility cues from the text, and re-ranks the candidate with new weights. The refreshed match score is what the persona's reciprocal-approval probability reads at decision time. What a user reveals mid-conversation directly influences whether they end up matched.

The challenger. Pure cosine retrieval. Top three by similarity. No features, no policy, no filters, no learning.

Dimension	Champion (feature pipeline)	Challenger (cosine-only)
Top-3 score distribution	0.45-0.75 typical — calibrated to feature explanation	0.78-0.92 — artificially high (cosine is similarity, not compatibility)
Feature explainability	Per-match reasons + compatibility breakdown	None
Cross-style bleed (couples seeing solos etc)	0% (hard filter)	25-40% (cosine ranks across all axes)
Same-gender enforcement (solo)	100% when gender is set	None
Adapts to in-chat signals	Yes — re-ranks per turn	No (frozen at retrieval time)
Adapts to revise feedback	Yes — text-feedback path with decay	No
Reciprocal-approval calibration	Score → probability is honest; observed approval rates 50-65%	Cosine → probability is dishonest; uniformly high observed approval (85-92%), deny meaningful only on outliers

The calibration story is what matters here. The champion's match score can be used directly as the reciprocal-approval probability — a 0.6 means a 60% chance the matched persona accepts. This makes denial *meaningful*. A low score honestly produces a no. The challenger's cosine score, used as a probability, produces uniformly high approval rates that hide compatibility signal in the noise.

The pool-doubling firedrill. When I first shipped the same-gender hard filter, top-three match scores within each gender dropped to around 0.43. Plugging that into `p_approve = match_score` meant a 57% deny rate on the reciprocal-approval roll. Users started reporting "I'm getting more denials than before." They were right. The filter had halved the effective candidate pool — 96 personas became 48 within their gender — and the top three within a smaller pool are necessarily lower-scoring. I had two options: fudge the probability formula (dishonest, breaks the calibration), or double the pool. I doubled the pool. Bumped `PERSONAS_PER_SLOT` from one to two in the seed matrix, ran the seed with `--resume` so existing personas weren't regenerated, generated 96 new personas. Top-three scores within each gender lifted 10-20 points; deny rates dropped proportionally; calibration stayed honest. Real cost: ~\$4 in language model and image generation calls.

Trade-offs. The champion ships with equal-weight priors across features. One over N for N features. This is the honest position given that I haven't yet earned confidence in per-feature importance from data, but it leaves measurable signal on the table. The infrastructure to learn proper weights is in place: every shown candidate, every accept, every reject is logged with the full feature breakdown at the time of the action. Phase two will compute replacement gradients — the difference between accepted and rejected feature vectors — and learn weights from accumulated events.

3.4 The pattern across all three champions

Look at the three approaches together and there's one architectural decision that shows up in all of them and that I'd defend more loudly than anything else in the system: **information starvation as quality control**. The persona-inferring language model never sees the embedding vector or the GoEmotions output it will be merged with downstream. The validator never sees the user prompt the reply was responding to. The matcher's policy file never sees individual user data, just the feature names to score. Each component gets exactly the slice of context relevant to its narrow task and the merge happens downstream, not co-prompted upstream. This is what keeps the system's outputs defensible. A language model with the answer key in front of it writes to that key. A language model that has to guess writes something honest.

4. How it actually runs

4.1 Deployment shape

Sonder is deployed across four service planes:

Edge and frontend. The React single-page application is served from a global content-delivery network. A catch-all edge function proxies every backend call to the application server, keeping the frontend on a single domain — required for the web-push service worker to register, and prevents the cross-origin tokens from being leaked. This wasn't the original plan. I tried to do it with simple HTTP redirects in the rewrite rules first; the platform refused them because cross-origin proxying through redirects is against its terms. Spent an evening on the edge-function rewrite. Worth it because the SPA stays on one domain and the API key for the backend stays out of the bundle.

Application server. A FastAPI process on a long-running container. The single process exposes REST routes, server-sent-event streams for itinerary generation and revision, websocket endpoints for chat and notifications, and runs the synthetic-agents background loop in the same lifespan. Designed to scale to multiple replicas with one configuration change — the in-memory websocket connection manager has to become a Redis pub/sub channel so messages sent to one container reach websocket sessions on another. The config variable is already in place; the swap is an evening, not an architecture migration.

Language model providers and the routing layer. A multi-provider routing engine reads a per-tier provider preference. Small tier (chat, openers, classifiers, *"why this?"* explanations, social posts, proposal evaluation) picks a primary; large tier (itinerary generation, complex refinement) picks another primary; either tier's automatic fallback is the other provider. Each provider client carries its own model identifier so cross-provider failover can never accidentally send an Anthropic model id to OpenAI or vice versa. This sounds obvious now but it bit me once during a Sonnet deprecation week.

Data stores. A managed vector index holds three namespaces — destinations, activities, candidate travellers. An operational document store holds user profiles, generated itineraries, chat sessions and messages, social posts, and the shared-itinerary negotiation log. Object storage holds binary assets — persona avatars, cached voice MP3s. Voice cache keys are SHA-256 of (text, voice_id) so a re-played message is free.

4.2 The full model inventory

Sonder orchestrates eight distinct generative or representational models, each chosen for a specific task profile rather than as a single one-size-fits-all:

Model	Pr ovi de r	Role
Claude Haiku 4.5	Ant hro pic	Small-tier conversational surfaces: chat replies, openers, classifiers, <i>"why this?"</i> explanations, social-post and open-trip-note generation, proposal evaluation
Claude Sonnet 4.6	Ant hro pic	Large-tier generative surfaces: itinerary generation, complex refinement, conflict resolution between paired travellers
GPT-4o-mini	Op en AI	Small-tier fallback when Anthropic is unavailable
GPT-4o	Op en AI	Large-tier fallback
text-embedding-3-small	Op en AI	All retrieval embeddings (1536-dim) — destinations, activities, traveller profiles, persona text, GoEmotions anchor vectors

Model	Provider	Role
gpt-image-1	OpenAI	Synthetic persona portrait generation, seed-time only
eleven_multilingual_v2	ElevenLabs	Persona voice text-to-speech, with deterministic voice-ID assignment per persona based on appearance → accent → gender lookup
GoEmotions cosine classifier	In-process	Emotion scoring over free text via cosine distance against 27 anchor vectors embedded once at process start

The reason there are eight rather than two: each does a thing the others can't do as well or as cheaply. Haiku handles persona-voiced texting at 25× lower cost than Sonnet with no quality loss against the validator stack. Sonnet's 16k output token ceiling matters for full-trip JSON. gpt-image-1's painterly outputs are explicitly biased away from photorealism, which is right for the *"Sonder Curated"* disclosure. ElevenLabs gives the persona voice library multilingual coverage that no general-purpose TTS matches. The GoEmotions cosine classifier lives in the same embedding space as everything else, so adding it cost zero new infrastructure.

4.3 Prompts live in code

Every persona-voiced prompt — chat reply, opener, proposal evaluator, social post, open-trip note, itinerary refinement, validator critics — is a module-level constant in the codebase. Versioning is by git. A prompt change is a reviewable commit. There is deliberately no external prompt store. The coupling between prompt and surrounding code is explicit, and a prompt change that breaks downstream parsing is caught by code review rather than discovered in production.

For larger structural prompt changes, the rollout pattern is to deploy the prompt change paired with a post-hoc normaliser that catches drift from outputs still in flight. The chat opener's *Hey {Name}!* greeting contract is the canonical example: when the format changed mid-deploy, both code paths gained the new prompt rules *and* a wide drifted-greeting matcher catching outputs from personas that hadn't yet observed the new prompt. Belt-and-braces because language models are stateless and you can't roll them mid-stream.

4.4 The five metrics watching for drift

Every language model surface in Sonder is instrumented through a single analytics layer. Five top-level metric families drive the dashboard:

User satisfaction. Match found, match approved, match denied, match regenerated, itinerary revision applied, refinement attempt counts. A spike in denies-per-match is a leading indicator of a calibration regression. A spike in revisions-per-itinerary is a leading indicator of an itinerary-quality regression.

Retrieval quality. Retrieval completion events carrying destination count and activity count. A surface returning zero candidates points at a corpus coverage gap before users start abandoning sessions.

Response quality. Validator stack executions per surface, with first-try approval rate, repair count, total latency, and the continuous semantic-genericity score. The genericity score is the most useful single metric in the whole stack because it's a continuous signal — a creeping rise in mean genericity over weeks is a soft drift warning before any individual reply looks bad.

Hallucination rate. Itinerary and persona validation pass rates broken down by issue category — assistant-voice leakage, memory contradiction, taxonomy leakage, etc. A category climbing in isolation points at a specific regression — an uptick in memory-contradiction issues after a chat-history cap change, for example.

Itinerary completion funnel. Plan started → trip generated → trip saved → trip viewed. The conversion between adjacent steps is the product's primary growth metric.

A per-feature distribution observer on the ranker side records mean, variance, and count per (surface, feature). This catches silent scale domination — for example, if raw cosine score is winning 80% of the combined ranker output because its distribution sits at 0.78 while ordinal fits sit at 0.5, that's visible in the aggregate rather than discovered three months later through empty engagement metrics.

4.5 The feedback loops in production

Three feedback paths are running today:

Implicit accept-reject events. Every shown match, every accept, every reject is logged with the candidate's full feature breakdown at the moment of the action. This is the substrate for phase-two gradient learning.

Explicit free-text revision feedback. When a user revises a generated itinerary, the feedback is classified for scope and target (which days, which categories), then routed through either a day-targeted regeneration prompt or a full-itinerary regeneration. The free text is also keyword-mapped to ranker features for that user — saying "*cheaper*" boosts budget-fit weighting going forward, with decay so repeated similar feedback doesn't oscillate the weights.

Live chat signals. The sarcasm-aware signal scanner runs after every message, re-ranks the candidate, updates the match score. The persona's reciprocal-approval probability reads the live score at decision time. So what a user reveals mid-conversation directly influences whether they end up matched. This is the loop I'm most excited about because it makes the system feel responsive in a way that's hard to articulate to a stakeholder until they experience it.

Human-in-the-loop is currently lightweight. I review validator-flagged samples through the analytics dashboard weekly. Prompt changes ship through git review with at least one other set of eyes. A formal moderation queue is phase-two work — synthetic content doesn't need it, but user-generated journal entries and feed posts will at scale.

5. What I learned, what's broken, and where this goes

5.1 What worked

Information starvation is the single biggest architectural lever I've found. The discipline of giving each component exactly the slice of context relevant to its narrow task — and merging downstream rather than co-prompting upstream — is what makes Sonder's outputs surprising rather than averaging-to-the-mean. The persona inference, the validator, the matcher's policy declarations, the synthetic-persona blind-writer seeding, they all use the same pattern. The discipline carries.

Async edit-in-place validation is the chat-UX win nobody is talking about. Small-model-first plus async-repair gives users large-model-validated quality at small-model latency. Three times faster, twenty-four times cheaper per reply, and users don't experience the seam. I'd recommend this pattern to any team running conversational AI in production.

Equal-weight priors with logged-everything infrastructure is the honest move when you don't yet have data. Refusing to hand-tune ranker weights and instead instrumenting every feature observation to disk is what keeps the door open for phase-two learning. Most products tune weights by gut and then can't reconstruct why. Sonder waits.

5.2 What's broken or limited

Equal-weight priors leave measurable signal on the table. Honest acknowledgement of a known gap. Resolution is engineering work — accumulate enough feedback events to compute replacement gradients, ship the learning loop. Not a research problem.

Synthetic-only safety filters. The same-gender hard filter is enforced against synthetic persona metadata. Real users joining the matching pool would need a verified gender field. Currently self-reported via the backfill prompt. Scale-up to real-user matching needs identity verification, and that's an entire phase-two product surface.

No automated test suite for LLM-dependent outputs. Local deterministic tests cover routing, validators, ranking math, pipeline shapes. The LLM-dependent surfaces are monitored through production telemetry rather than CI-tested, because LLM outputs are hard to assert against deterministically. This is a deliberate trade-off but it means regressions can ship and only get caught by aggregate metric drift. Mitigating with the validator stack telemetry is the current answer; building a sample-evaluation harness with LLM-as-judge is a phase-two consideration.

Vector store corpus maintenance is manual. Fifty curated destinations and five hundred curated activities is a curation budget I'm carrying personally. A user picking a city not in the corpus falls back to a "invent plausible activities for {city}" prompt that re-introduces the exact hallucination risk the RAG pipeline normally controls. Automated corpus expansion is phase two: scrape candidate venues from public sources, embed on-demand, validate against the same critic stack. Estimated effort: two engineering weeks plus an ongoing curation review queue.

Synthetic content doesn't auto-throttle. The synthetic-agents loop runs at the same cadence regardless of real-user density. Once real-user content reaches critical mass, the synthetic side should throttle down. Right now it's a manual configuration change. A self-balancing implementation is straightforward — track new-content-per-hour by source — but I haven't shipped it yet because the platform isn't there yet.

5.3 Where this goes

- **Phase-two ranker learning.** Replace equal-weight priors with gradient-learned weights from accumulated accept-reject events. The data substrate is already in place.
- **Automated activity-corpus expansion.** Scraping plus validator-gated embedding for novel destinations. Removes the prompt-fallback hallucination risk.
- **Multi-modal persona reveal.** The cinematic destination-reveal screen is currently photo-driven; phase two could add persona-voiced audio greetings (the voice infrastructure already exists for chat) or short generated video.
- **Layered prompt-injection defence.** The current input sanitiser is a lightweight regex first-line. A language model classifier on top for high-stakes free-text fields (proposals, chat, journal) is the obvious second layer.
- **Cross-candidate diversity in ranking.** The matcher's reranker stage is declared as a no-op in v1, reserved for diversity / fatigue / sequencing features. Maximum marginal relevance would prevent the top three matches from all being near-duplicates.
- **Verified identity for real users.** Self-reported gender is a starting point, not a scale answer.

5.4 The sources I drew from

Models used in production.

- Anthropic Claude — Haiku 4.5 for small-tier conversational surfaces, Sonnet 4.6 for large-tier generation, validator tier available on either
- OpenAI — GPT-4o-mini and GPT-4o (fallback provider, same tier split)
- OpenAI text-embedding-3-small — 1536-dimensional embeddings, used uniformly across every retrieval surface
- OpenAI gpt-image-1 — synthetic persona portrait generation, seed-time only
- ElevenLabs eleven_multilingual_v2 — persona voice TTS
- GoEmotions 27-label classifier — in-process cosine over anchor vectors, lives in the shared embedding space

Datasets and curated corpora.

- 192 LLM-designed synthetic solo travellers and 18 couples, all seeded into the vector store
- Hand-curated destination corpus (~50 cities) and per-destination activity corpora (~500 activities total)
- Closed twelve-dimension push-pull persona vocabulary
- Closed eight-key emotional-signature taxonomy with tone-anchored glosses
- GoEmotions 27-label emotion vocabulary

Academic frameworks the system is built on.

- Dann, G. (1977). "Anomie, Ego-Enhancement and Tourism." *Annals of Tourism Research*, 4(4), 184-194. — Original Push-Pull Motivation Theory in travel research.
- Crompton, J. L. (1979). "Motivations for Pleasure Vacation." *Annals of Tourism Research*, 6(4), 408-424. — Extension of Dann's framework that informed the six-and-six dimension adaptation.
- Demszky, D., et al. (2020). "GoEmotions: A Dataset of Fine-Grained Emotions." *arXiv:2005.00547*. — Source of the 27-label emotion taxonomy used as anchor vectors.

External APIs and services.

- Wikipedia REST API — destination context + lede paragraphs + infobox imagery
- Pixabay — popularity-ranked travel photography for the cinematic reveal
- OpenWeather — current weather by lat/lon
- Nominatim / OpenStreetMap — geocoding with 1-req/sec etiquette
- ExchangeRate API — currency conversion with 30-currency hardcoded fallback

Infrastructure.

- Pinecone — managed vector index, three namespaces
- Firestore (named-database mode) — operational document store
- Firebase Authentication, Firebase Storage
- Render — application server hosting
- Cloudflare Pages + edge functions — frontend hosting and backend proxying
- VAPID + service worker — offline web push notifications

The repository where the work lives. github.com/shreyast36/sonder

Built over a series of sleepless nights by someone who got tired of explaining their trip to an algorithm.